

**Octopus training stack:
JAX side by side with SB3**

Pulak Mehrotra

May 2026

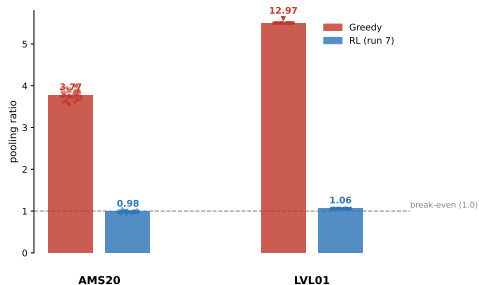
The slow part is the simulator (CPU), not the neural net.

Realistic run (reward A, many traces,
augmentation, `n_envs=32`)

What we measure	Typical
Time for 2M steps	≈ 10.5 h
Env steps per second	~ 50 – 56
GPU busy during train	$< 10\%$
Long reset when cache misses	~ 120 ms

Takeaway: time goes into `env.py`
(reward, observations, bookkeeping) —
not PyTorch SAC.

RL quality: SB3 stays the trusted path for results.



Each dot is one pod mapping (n = 50 random seeds). Greedy LVL01 bar capped at 5; true mean shown.

vs. baselines (pooling savings)

RL is the only rule that lands near break-even on **both** unseen traces; greedy does badly.

What JAX changes:

not the maths of the problem
— only *how many steps per hour* we can run.

Two ways to train: keep SB3, add JAX.

- ▶ **Baseline (trust this for papers): Gym + Stable-Baselines3 SAC** — traces, augmentation, rewards, logging, offline eval hooks stay as today.
- ▶ **Parallel path (for speed): JAX** — simulator state as plain arrays, `jit` stepping, bulk rollouts with `scan` and `vmap`.
- ▶ **Both see the same world:** same observations, same rewards (A/B/current), same episode setup — so curves stay comparable.
- ▶ **Why JAX at all?** multiprocessing helps a lot, but Python still runs every env step. JAX pushes the stepping loop into compiled code so we can chew through timesteps cheaper.

Put the simulator in plain arrays (numpy now, JAX later).

JAX packs the same numbers into one grouped object (pytree):

```
(mpd_load, dealloc_buf, host_*, mpd_dt/mpd_mem/mpd_n, rng_key,
      indices)
```

Concept: one contiguous row per pool (numpy layout), grouped for jit.

bank	<i>Packed row</i> (<code>_mpd_dt</code> , <code>_mem</code> , ...)
-------------	---

0	$\Rightarrow$ fixed slots $\times$ VMs per pool $\rightarrow$ scalar <code>_mpd_n[0]</code>
---	---

$\vdots$	... analogous rows $\rightarrow$ <code>_mpd_n[1]</code> , ... <code>_mpd_n[K-1]</code>
----------	--

Not a nested list of Python tuples per alive VM — arrays + counts.

Where Python stops and JAX starts each episode.

Episode shape: host builds tensors once, then GPU steps in a compiled loop.

Host (CPU, Python)

`_generate_pod`
pick hosts for this episode

`_build_events`
filtered VM arrivals in time order

Pad $\rightarrow$ one big table
(`max_events` $\times$ 4) with blank rows marked

Device (jit)

`device_put`

`scan(step) + vmap`
bulk stepping

once per episode

Sizing example: MAX_TICKS=2304; up to 256 live VMs per pool slot

Same three stages; only the implementation changes.

Each numbered row means the same job on both sides.

Stage	SB3 + Gym + PyTorch	JAX + Optax (+ XLA)
1. Roll-out & gather	SubprocVecEnv (spawn): numpy OctopusMemPoolEnv.step() per worker; observations to CUDA. $\approx$ same tuples	Compiled stepping: <code>vmap</code> $\times$ <code>lax.scan(stepfn)</code> . Arrays everywhere; RNG split per chunk; compiled inner loop. no Python in the stepping hot path
2. Train policy	SAC (Stable-Baselines3): replay $\rightarrow$ <code>.learn()</code> minibatches $\rightarrow$ PyTorch on GPU. same algorithm family	SAC (ported): replay on CPU for now $\rightarrow$ one jit block for losses + optimizer; actor, twin critics, slow targets, auto α .
3. Log & test	Today: TB/W&B, checkpoints, eval callbacks; extra GPU worker scores pooling. same dashboards / checkpoints	Todo: checkpoints; Gym & JAX episode parity tests; wire pooling scorer later.

What gets compiled; what we want to measure.

Rough throughput picture

Goal in speedup notes: finish 2M steps in ≤ 3 h $\Rightarrow \sim 185+$ steps/s env-side.

Situation	Rate	Comment
Heavy config (meas.)	$\sim 53/\text{s}$	reward A etc.
Sketch / light path	$\sim 1221/\text{s}$	32 workers, simpler inner loop
Faster numpy env	TBD	vectorise + caches
JAX full stack	TBD	full stack & IO wired